



E Authentication System Using QR Code & OTP

A MINI PROJECT REPORT

By
Malak Al Hakamani

Table of Contents

I. Abstract.....	3
II. Keywords.....	3
III. Innovative Idea / Concept.....	3
IV. Introduction.....	3
V. System Background.....	4
VI. Authentication Methods (Types).....	4
VII. Objectives.....	4
VIII. Scope and Limitations.....	5
IX. Project Timeline.....	5
X. System Analysis.....	6
XI. Implementation.....	8
XII. Explanation of Key Functions.....	9
XIII. Tools and Technologies Used.....	9
XIV. Security Analysis.....	10
XV. Common Security Threats.....	10
XVI. Comparison: OTP vs Traditional Passwords.....	10
XVII. Example Output.....	11
XVIII. Future Work.....	11
XIX. Conclusion.....	12
XX. References.....	12

I. Abstract

This project demonstrates a secure authentication system using QR Code and One-Time Password (OTP) for better access control. It creates a random and secure OTP using Java and converts it into a QR Code (simulated) for authentication. The user is required to scan the QR Code and input the OTP to authenticate themselves.

This approach enhances security by providing access to the system to only those who have the temporary password. Using OTPs eliminates threats like password stealing, replay and spoofing. This project showcases the application of practical concepts of cybersecurity, including authentication and access control using Java.

II. Keywords

OTP, QR Code, Authentication, Java, Cybersecurity, Secure Access, SecureRandom, Access Control

III. Innovative Idea/Concept

The project's innovative concept is integrating OTP authentication and QR Code display. Rather than showing the OTP as text, it is represented by a QR Code.

This method provides a higher level of security by minimising exposure of the OTP, and mimics how OTP is used in real life banking, email and other secure applications.

IV. Introduction

In cybersecurity, authentication is a critical process to allow access to resources. Conventional password-based authentication is prone to attacks like phishing, brute force, and password reuse.

In this project, we propose a secure method of authentication using One-Time Password (OTP) and QR Code. It uses the SecureRandom class in Java to generate a random OTP, which is then converted into a QR Code (simulated) and verified by the user.

OTP systems add an extra layer of security as the password is valid only for one transaction. This project showcases authentication in systems like banks and secure website login.

The next section explains about the background ideas used in this system because secure authentication is so important.

V. System Background

One-Time Password (OTP) is a popular authentication technique in cybersecurity. This is a short-lived, one-time-use password that is used to authenticate a user for a particular session or transaction. OTP is a one-time password that cannot be used again, making it a more secure alternative to traditional passwords, which are vulnerable to replay attacks and password stealing.

In this approach, OTP is generated using a secure random number generator. This makes it hard for any hacker to predict or duplicate the password.

The QR Code is used to encode data. QR Codes are widely used in authentication, payments and data sharing in real-world applications. In this project, the OTP is converted to a QR Code (simulated) format, providing an additional level of security by obscuring the password.

User authentication is the process of confirming the user's identity. Today systems employ multi-factor authentication (MFA) and OTP serves as the second authentication factor. This project is a simple simulation of such systems using OTP and QR Code for authentication.

VI. Authentication methods (Types)

Authentication can be classified into different types. The most common is single-factor authentication, which relies only on a password. However, this method is weak because passwords can be stolen or guessed.

More secure systems use multi-factor authentication (MFA), which combines multiple factors such as:

- Something the user knows (password or OTP)
- Something the user has (mobile device or QR code)
- Something the user is (biometric data like fingerprint)

- In this project, OTP acts as a second layer of security, making the system closer to real-world authentication systems.

VII. Objectives

- *General Objective:*

To create a secure user authentication system using One-Time Password (OTP) and QR Code technology for better security.

- *Specific Objectives:*

- To generate secure One-Time Passwords using Java.
- To simulate QR Code representation of OTP.
- To implement user authentication using OTP verification.
- To demonstrate secure login mechanisms in cybersecurity.
- To reduce risks of unauthorized access and password reuse.

VIII. Scope and Limitation of the Mini Project

- *Scope:*

This project is about creating a secure login mechanism for OTP with QR Code representation. It illustrates simple cybersecurity principles, implemented in Java, as a teaching tool.

- *Limitations:*

- The system simulates a QR Code, but a real QR Code is generated externally of the system to show how it works in practice.
- No SMS/email OTP delivery system.
- No database integration.
- Console-based application only.
- None of the OTP expiration timers put in place.

IX. Project Timeline (Table & Ganntt Chart)

Task	Week 1	Week 2	Week 3	Week 4
Requirement Analysis	✓			
System Design	✓	✓		
Coding Implementation		✓	✓	
Testing & Debugging			✓	
Report Writing				✓
Final Review				✓

Task	Week 1	Week 2	Week 3	Week 4
 Analysis				
 Design				
 Coding Implementation				
 Testing & Debugging				
 Report Writing				
 Final Review & Submission				

X. System Analysis

The system is set up as a simple way to verify identity using OTP. It starts by making a random OTP in a safe way, which makes sure that it is safe and hard to guess.

The system then shows the user a QR Code version of the OTP that was made. The user scans the QR code and types in the OTP.

The system checks the entered OTP against the one it made. If the two values are the same, access is granted; if they are not, access is denied.

This method makes sure that privacy and integrity are protected by checking the user's identity before giving them access. This is a key requirement in secure authentication systems.

1. SYSTEM OVERVIEW

The system is a secure authentication application that uses One-Time Password (OTP) and QR Code technology. The system generates a random OTP, encodes it into a QR Code (simulated), and displays it to the user. The user scans the QR Code and enters the OTP. The system then verifies the OTP and grants or denies access based on the result.

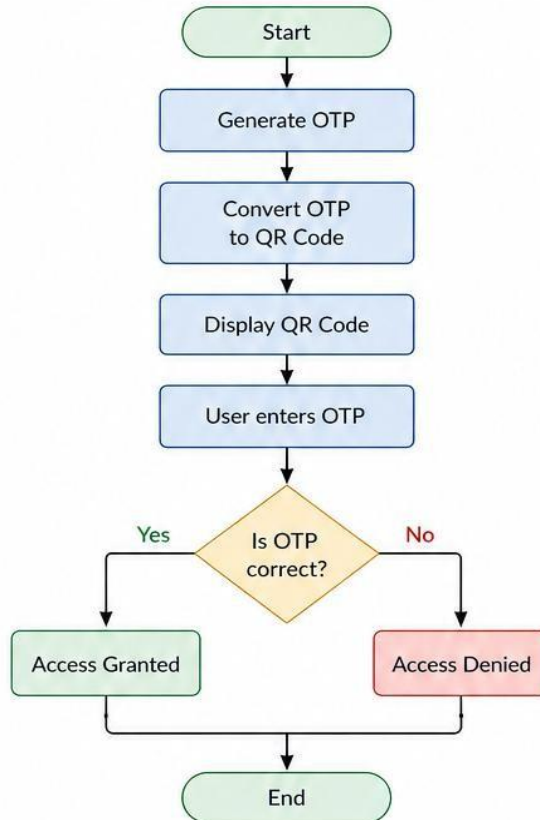
2. INPUTS

- User request to generate OTP
- OTP entered by user (from scanned QR Code)

3. OUTPUTS

- Display QR Code (containing OTP)
- Authentication Result (Access Granted or Access Denied)

4. SYSTEM FLOW (FLOWCHART)



5. PROCESS DESCRIPTION

1. The system generates a secure random OTP.
2. The OTP is converted into a QR Code format.
3. The QR Code is displayed to the user.
4. The user scans the QR Code and enters the OTP.
5. The system verifies the OTP.
6. If the OTP is correct, access is granted; otherwise, access is denied.

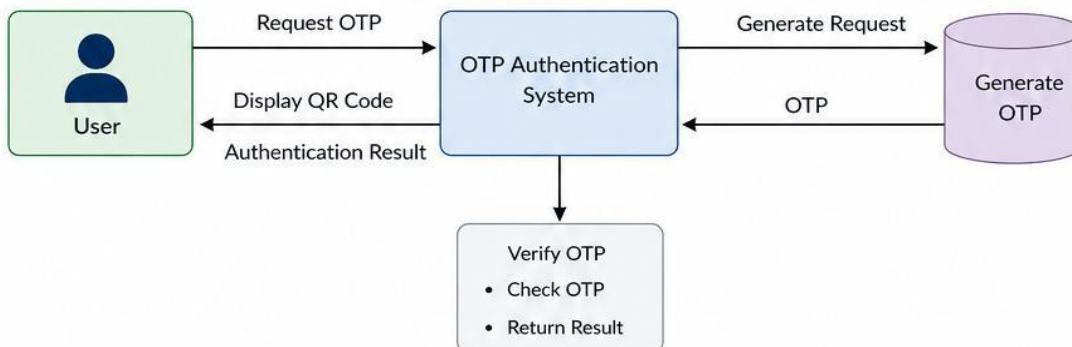
6. ACTORS



User

- Requests OTP and enters OTP for authentication.

7. DATA FLOW DIAGRAM (DFD - LEVEL 0)



The next section explains how the system is built using Java after looking at its design and features.

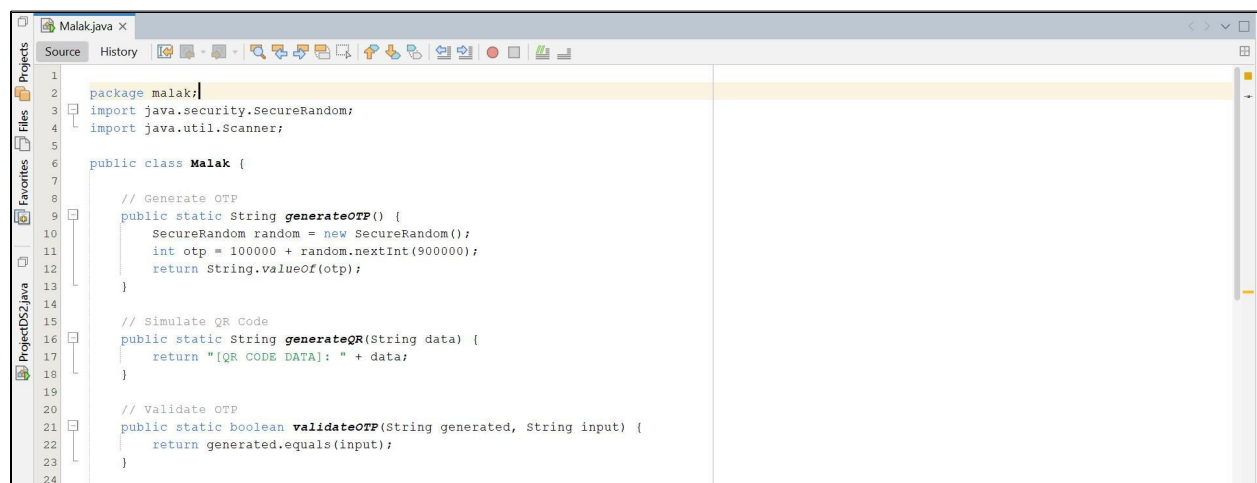
XI. Implementation (Java Code)

Java is used to build the system because it is easy to use and has built-in security libraries.

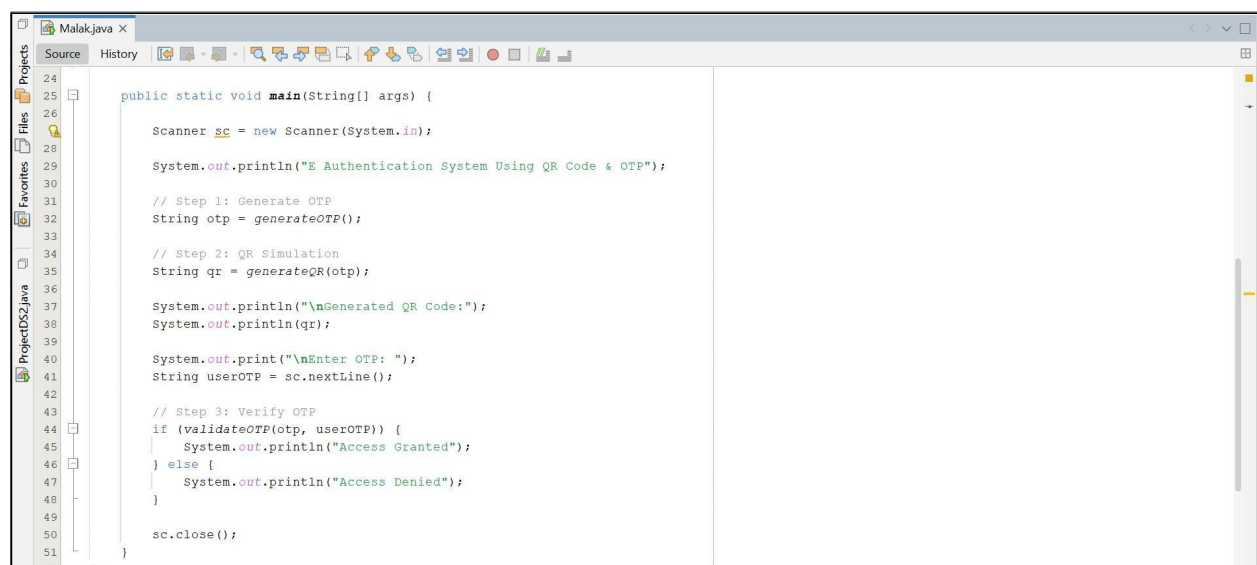
The implementation is done in steps. The `SecureRandom` class is used to make a random OTP. Then, the OTP is changed into a QR Code format (which looks like text output). The system then asks the user to type in the OTP.

Lastly, the system checks the entered OTP against the one it made to see if access should be granted or denied.

This implementation shows how to use basic programming techniques to add authentication logic while still following security rules.



```
1 package malak;
2
3 import java.security.SecureRandom;
4 import java.util.Scanner;
5
6 public class Malak {
7
8     // Generate OTP
9     public static String generateOTP() {
10         SecureRandom random = new SecureRandom();
11         int otp = 100000 + random.nextInt(900000);
12         return String.valueOf(otp);
13     }
14
15     // Simulate QR Code
16     public static String generateQR(String data) {
17         return "[QR CODE DATA]: " + data;
18     }
19
20     // Validate OTP
21     public static boolean validateOTP(String generated, String input) {
22         return generated.equals(input);
23     }
24 }
```



```
24
25 public static void main(String[] args) {
26
27     Scanner sc = new Scanner(System.in);
28
29     System.out.println("E Authentication System Using QR Code & OTP");
30
31     // Step 1: Generate OTP
32     String otp = generateOTP();
33
34     // Step 2: QR Simulation
35     String qr = generateQR(otp);
36
37     System.out.println("\nGenerated QR Code:");
38     System.out.println(qr);
39
40     System.out.print("\nEnter OTP: ");
41     String userOTP = sc.nextLine();
42
43     // Step 3: Verify OTP
44     if (validateOTP(otp, userOTP)) {
45         System.out.println("Access Granted");
46     } else {
47         System.out.println("Access Denied");
48     }
49
50     sc.close();
51 }
52 }
```


XII. Explanation of Key Functions

1. generateOTP():

Creates a safe 6-digit One-Time Password with SecureRandom.

2. generateQR():

Mimics QR Code generation by coded OTP as readable information.

3. validateOTP():

Makes comparisons between generated OTP and user input to authenticate.

4. SecureRandom:

Gives cryptographically secure random number generation.

5. Scanner:

Handles user input from the console

XIII. Tools and Technologies Used

The project is implemented in the programming language Java, which has native libraries for secure programming.

- The main tools include:

- Java (JDK)
- Scanner
- IDE (NetBeans)
- SecureRandom: for generating secure OTP values

XIV. Security Analysis

The system's security is improved by the use of One-Time Passwords (OTP) that are valid for a single use only. This helps avoid replay attacks and decreases the risk of fraudulent access.

Using SecureRandom, the OTP values are generated in an unpredictable way. It also helps avoid direct exposure of the OTP using QR Code representation.

XV. Common Security Threats

There are various threats to even basic authentication systems including:

- Brute-force attacks (guessing OTP)
- Phishing attacks (tricking users)
- Replay attacks (reusing OTP)
- Man-in-the-middle attacks (intercepting data)

These attacks are avoided by OTP as it is temporary and cannot be reused.

XVI. Comparison: OTP vs Traditional Passwords

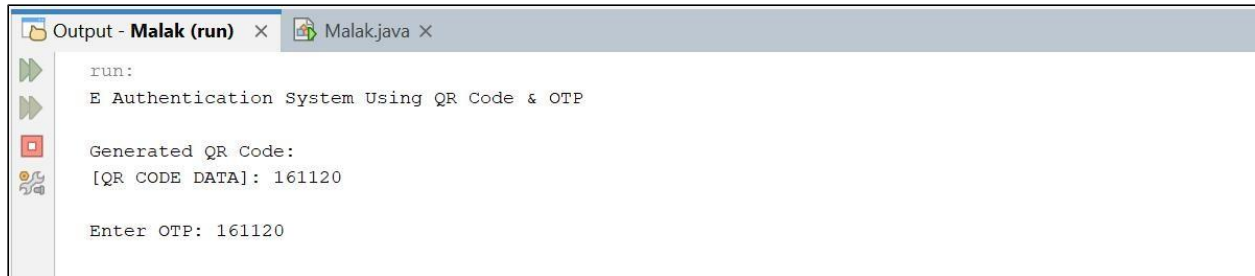
Feature	 Traditional Password	 OTP (One-Time Password)
Reusability	Can be reused	One-time use only
Security Level	Lower	Higher
Risk of Theft	High	Low
Validity Duration	Long-term	Temporary
Usage	Static login	Dynamic authentication
Protection Against Replay Attacks	Weak	Strong

XVII. Example Output

The output below shows what the authentication system did after generating the OTP.

The system generates a random OTP and shows it in the console as simulated QR Code data.

Example :



```
run:
E Authentication System Using QR Code & OTP
Generated QR Code:
[QR CODE DATA]: 161120
Enter OTP: 161120
```

An external QR Code generator was used to turn the generated OTP into a real QR Code to show how it could be used in the real world.

The user can scan this QR Code to get the OTP and then enter it into the system for authentication.



XVIII. Future Work

These improvements would make the system closer to real-world authentication systems.

- *Future improvements may include:*
 - Using real QR Code generation libraries
 - Adding OTP expiration time
 - Sending OTP via SMS or email
 - Integrating a database system

Based on the analysis and implementation that were talked about, the following conclusion sums up what happened with the project as a whole.

XIX. Conclusion

The project successfully implemented a secure authentication system with OTP and QR Code. It demonstrates the implementation of the latest cyber security technologies using basic Java programming.

The system provides an access control mechanism that ensures only legitimate users can access resources by using a temporary password, emphasizing the need for authentication and access control in secure systems.

XX. References

- NIST Digital Identity Guidelines: <https://pages.nist.gov/800-63-3/sp800-63b.html>
- Oracle SecureRandom: <https://docs.oracle.com/javase/8/docs/api/java/security/SecureRandom.html>
- Authentication Guide by OWASP: https://cheatsheets.owasp.org/cheatsheets/Authentication_Cheat_Sheet.html.
- Java OTP GeeksForGeeks: <https://www.geeksforgeeks.org/generating-otp-one-time-password-in-Java/>